

POLITECNICO DI TORINO

Master's Degree in Cybersecurity



Master's Degree Thesis

Penetration Test in Automotive Systems

Supervisor:

Prof. Alessandro Savino

Candidate:

Dennis D'Amico

Internship Tutor

TurinTech SE:

Eng. Mauro Bruno

Academic year 2025/2026

Abstract

LOREM ipsum

LOOK content/abstract.tex

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Prof. Alessandro Savino, for the time dedicated.

Special thanks goes to my company tutor, Ing. Mauro Bruno, and the entire team at TurinTech for their constant support.

Lastly, I am endlessly grateful to my family and friends for their company along the journey and for always cheering me up and believing in me even when I couldn't.

Table of Contents

1	Introduction to Automotive system and structure	1
1.1	Evolution of Automotive Architecture	1
1.1.1	Functional Domains and Network Requirements	1
1.1.2	Structural Limitations and Protocol Evolution	2
1.1.3	Convergence and Automotive Cybersecurity Implications	2
1.1.4	Rationale for CAN Protocol Analysis	3
1.2	CAN Communication Protocol	4
1.2.1	CAN Data Frame Structure	4
1.2.2	Security Properties and Architectural Vulnerabilities	5
2	Chapter Two: Unified Diagnostic Services Protocol	7
2.1	Overview of ISO 14229 Standard	7
2.2	Client-Server Diagnostic Model	7
2.3	Key UDS Services and Request/Response Formats	7
2.4	Diagnostic Session Control	7
2.5	Security Access Mechanisms	7
3	Chapter Three: Threat Modeling and Vulnerability Analysis	9
3.1	Automotive Threat Modeling	9
3.2	UDS protocol vulnerabilities	9
3.3	Attack Vectors on the Diagnostic Bus	9
4	Chapter Four: Experimental Laboratory Setup	11
4.1	Hardware Testbed Architecture	11
4.2	Software Environment	11
5	Chapter Five: Implementation of Penetration Testing and Attacks	13
5.1	Sniffing on CAN interface	13
5.2	Fuzzing on CAN protocol	13
5.3	Message injection	13
5.4	UDS Diagnostic Discovery	13
5.5	Seed Entropy	13
5.6	Replay Attack on Security Access	13
5.7	Parameter Discovery on Security Access	13

6 Chapter Six: Mitigation Strategies and Conclusions	15
6.1 Secure Diagnostics via SecOC	15
6.2 Intrusion Detection and Prevention Systems	15
6.3 Final Remarks and Future Work	15
A Python Scripts	16
A.1 UDS Diagnostic Discovery	16
Bibliography	19
Dedications	20

List of Figures

1.1	Structure of a Standard CAN 2.0A Data Frame. (Source:[7])	4
-----	---	---

List of Tables

Acronyms

!!!	LOOK glossaries.tex.
ADAS	Advanced Driver Assistance Systems.
CAN	Controller Area Network.
CAN-FD	Controller Area Network - Flexible Datarate.
ECU	Electronic Control Unit.
E/E	Electrical and Electronic.

Chapter 1

Introduction to Automotive system and structure

1.1 Evolution of Automotive Architecture

For several decades the design of vehicles was mainly focused on providing a reliable mechanical infrastructure that would be both secure and economical. However with the advancement of embedded technology the paradigm shifted toward providing software-driven platforms that provide services to the user to enhance the commodity of the transportation.

To support this digital transition, the underlying Electrical and Electronic (E/E) architecture had to evolve through three main topological models[1]:

1. **Distributed architecture (The legacy model):** In early electronic systems, every new feature—such as anti-lock braking (ABS) or airbag deployment required a dedicated Electronic Control Unit (ECU). These units were in a closed network state and would not interact with external environments.
2. **Domain control architecture (The current standard):** In this division the architecture is organised into domains according to functionality. Typically, these domains include Powertrain, Chassis, Body, ADAS, and Infotainment.
3. **Zonal Architecture (The Next-Generation Paradigm):** The latest evolutionary step organises the vehicle's network based on physical location (e.g., front, rear, left, right) rather than logical function. Localised Zonal Control Unit collect data from nearby components and stream it to a central ECU, drastically reducing the amount of wiring necessary and allowing the use of strategic high speed wiring.

1.1.1 Functional Domains and Network Requirements

The current adopted model classifies ECUs based on the requirements for data throughput, timing determinism, and fault isolation [2]:

- **Safety-Critical Domains (Powertrain and Chassis):** Units that manage core functionalities such as the Engine Control Module (ECM) or the ABS need low latency, typically ranging from 1 to 10 milliseconds, and absolute time determinism. Because a delayed message could result in catastrophic failure, these zones require strict *fault isolation*. If an ECU fails, the communication protocol must alert the rest of the network.
- **High-Bandwidth Domains (ADAS and Telematics):** Advanced Driver Assistance Systems process continuous data streams from camera sensors and others real-time driver assistance systems. These systems require both high throughput and deterministic scheduling to perform safety-critical actions, such as automated braking, are executed without any delay.
- **Low-Criticality Domains (Body and Comfort):** This domain is responsible for non critical components such as climate control, seat adjustments and interior lighting. These networks operate on relaxed latency constraints and low bandwidth requirements, making them ideal for more economical protocols.

1.1.2 Structural Limitations and Protocol Evolution

The presence of these different functional domains quickly revealed severe structural limitations within early E/E architectures, incentivising the development of high-speed in-vehicle communication protocols:

- **Moving from Traditional CAN to CAN-FD:** The traditional Controller Area Network (CAN) bus has been the basis of automotive networking for decades. However its maximum data rate of 1 Mbps and payload limit of 8 bytes per frame became a critical bottleneck. As the number of onboard ECUs increased, networks suffered from severe traffic saturation. This led to the introduction of CAN Flexible Data-rate (CAN-FD) [3], which mitigates these limits by increasing data throughput up to 5 Mbps and expanding the payload up to 64 bytes per frame.
- **The Transition to Automotive Ethernet:** Despite the bandwidth improvements of CAN-FD, modern data-heavy applications require transmission speeds far beyond the capabilities of legacy fieldbuses. This demand has led the adoption of Automotive Ethernet (standardized under IEEE 100BASE-T1 and 1000BASE-T1) [4], which provides high-bandwidth, bidirectional communication (from 100 Mbps to over 1 Gbps) over a single unshielded twisted pair of copper wires.

1.1.3 Convergence and Automotive Cybersecurity Implications

This transition from isolated, low-bandwidth buses to high-speed, interconnected topologies has dramatically changed the automotive cybersecurity landscape. Traditionally, low critical domains were physically separated from safety-critical networks.

Nevertheless, producer requirements for over-the-air (OTA) firmware updates, remote cloud diagnostics, and sophisticated infotainment connectivity have forced these distinct domains to be linked via centralized gateways [1].

This architectural convergence leads to severe security vulnerabilities, which are now governed by the *ISO/SAE 21434 Automotive Cybersecurity Engineering standard* [5]. While traditional safety engineering ensures that a system is robust towards random hardware faults, cybersecurity engineering must ensure the containment of intentional, malicious exploits.

Because low-criticality domains like the Infotainment system possess external wireless interfaces (Bluetooth, Wi-Fi, Cellular V2X), they represent the primary point of entry for an attacker [6]. If the network lacks cryptographic boundary enforcement or strong gateway-level filtering, a malicious actor can inject unauthorized diagnostic payloads into a low-criticality bus. Attackers can then exploit internal routing mechanisms to perform a lateral movement and compromise safety-critical domains.

1.1.4 Rationale for CAN Protocol Analysis

Despite the rapid deployment of high-bandwidth solutions such as Automotive Ethernet, the CAN and CAN-FD protocols remain the de facto standards for real-time powertrain, chassis, and body control due to their low cost, robustness, and deterministic arbitration. Therefore, understanding the internal functionalities, intrinsic design limitations, and underlying vulnerabilities of the CAN protocol is essential for securing modern E/E architectures. The following section describes in detail its signalling mechanism, frame structure, and security weaknesses.

1.2 CAN Communication Protocol

The Controller Area Network (CAN) is a multi-master, asynchronous serial bus protocol designed by Robert Bosch GmbH in the 1980s to replace complex point-to-point vehicle wiring harnesses. It operates on a differential two-wire bus consisting of CAN High (CAN_H) and CAN Low (CAN_L) lines. Communication relies on two logical states defined by voltage differences: *dominant* (logical 0) and *recessive* (logical 1). Because a dominant state actively drives the bus voltage difference, it always overwrites a recessive state, enabling robust collision resolution without data corruption.

1.2.1 CAN Data Frame Structure

The CAN protocol transfers data across the network using specific frame types, the most notable being the standard Data Frame (defined in the CAN 2.0A specification). A standard CAN Data Frame is divided into distinct functional fields, each serving a specific role in synchronization, identification, data payload transmission, and error checking.

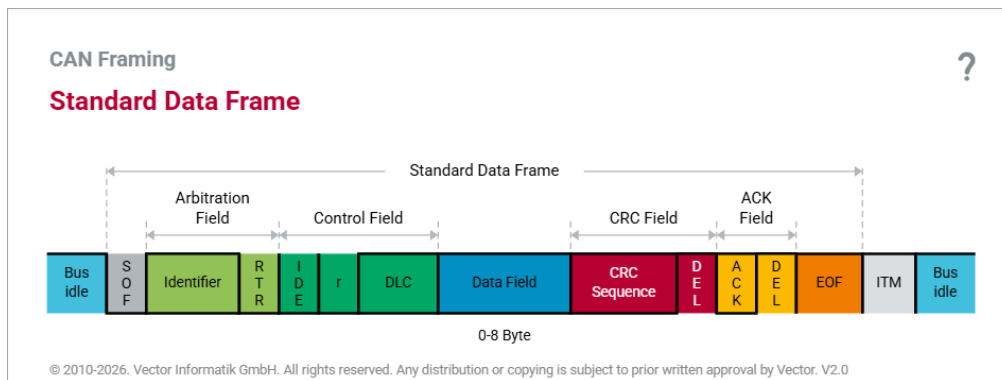


Figure 1.1: Structure of a Standard CAN 2.0A Data Frame. (Source:[7])

As illustrated in Figure 1.1, the standard Data Frame consists of the following sequential fields:

- **Start of Frame (SOF):** A single dominant bit (0) that synchronizes the clocks of all nodes on the bus after a period of idle network activity.
- **Arbitration Field:** Composed of an 11-bit Identifier (ID) and a single Remote Transmission Request (RTR) bit. The identifier defines the priority of the message; lower numerical ID values represent higher priority. The RTR bit distinguishes between a data frame (dominant 0) and a remote request frame (recessive 1).
- **Control Field:** A 6-bit field that includes the Identifier Extension (IDE) bit, a reserved bit (r0), and the 4-bit Data Length Code (DLC). The DLC specifies the exact number of bytes contained in the subsequent payload, ranging from 0 to 8 bytes.

- **Data Field:** The core payload of the message, containing from 0 to 8 bytes (0 to 64 bits) of application-specific parameters, such as sensor readings or control commands.
- **Cyclic Redundancy Check (CRC) Field:** A 16-bit field containing a 15-bit checksum derived from the preceding frame bits, followed by a single recessive CRC Delimiter bit. Nodes use this field to detect bit errors during transmission.
- **Acknowledgment (ACK) Field:** A 2-bit field consisting of the ACK Slot and an ACK Delimiter. The transmitting node sends a recessive bit during the ACK Slot; any receiving node that validates the frame integrity overwrites this slot with a dominant bit, confirming successful reception.
- **End of Frame (EOF):** A sequence of 7 consecutive recessive bits indicating the definitive termination of the Data Frame.

1.2.2 Security Properties and Architectural Vulnerabilities

When the CAN protocol was standardized, automotive networks were physically isolated, closed systems. Consequently, the protocol was designed to maximize dependability, real-time determinism, and hardware efficiency, completely omitting cryptographic security measures. Analyzing CAN through the lens of modern information security paradigms (the CIA triad) reveals critical structural deficiencies:

- **Lack of Authentication and Integrity (No Source Verification):** CAN frames lack any origin identifiers or cryptographic signatures. Messages are broadcast to the entire bus, and receiving nodes filter them based solely on the Message ID, which identifies the data type (e.g., engine speed) rather than the transmitter. As a result, any node connected to the bus can masquerade as another by injecting frames with identical IDs. The 15-bit CRC field only protects against random hardware bit-errors caused by electromagnetic interference, offering no protection against intentional malicious modifications.
- **Lack of Confidentiality (Open Broadcast Nature):** Because CAN uses a shared, broadcast medium, every node on the bus can read every transmitted frame. There is no native encryption layer at the data link or physical levels. An attacker who gains physical or logical access to the bus can perform passive eavesdropping, capture all network traffic, and reverse-engineer proprietary vehicle telemetry.
- **Vulnerability to Denial of Service (Arbitration Exploitation):** The deterministic priority mechanism, while beneficial for safety-critical real-time scheduling, introduces a major vulnerability. An attacker can continuously flood the bus with dominant bits or maximum-priority frames (ID 0x000). Because of the non-destructive bitwise arbitration, lower-priority legitimate traffic is perpetually blocked, resulting in a total network Denial of Service (DoS).

- **Susceptibility to Cyber-Physical Attacks (Error Handling Exploitation):** The native fault-confinement mechanism of CAN can be weaponized. When a node detects too many transmission errors, it automatically transitions from an *Error Active* state to an *Error Passive* state, and eventually into a *Bus-Off* state, which physically disconnects it from the network. Attackers can exploit this by precisely injecting dominant bits during the transmission of a target ECU, forcing it into a permanent Bus-Off state and effectively neutralizing a safety-critical component.

Chapter 2

Chapter Two: Unified Diagnostic Services Protocol

2.1 Overview of ISO 14229 Standard

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

2.2 Client-Server Diagnostic Model

2.3 Key UDS Services and Request/Response Formats

2.4 Diagnostic Session Control

2.5 Security Access Mechanisms

Chapter 3

Chapter Three: Threat Modeling and Vulnerability Analysis

3.1 Automotive Threat Modeling

3.2 UDS protocol vulnerabilities

3.3 Attack Vectors on the Diagnostic Bus

Chapter 4

Chapter Four: Experimental Laboratory Setup

4.1 Hardware Testbed Architecture

4.2 Software Environment

Chapter 5

Chapter Five: Implementation of Penetration Testing and Attacks

- 5.1 Sniffing on CAN interface
- 5.2 Fuzzing on CAN protocol
- 5.3 Message injection
- 5.4 UDS Diagnostic Discovery
- 5.5 Seed Entropy
- 5.6 Replay Attack on Security Access
- 5.7 Parameter Discovery on Security Access

Chapter 6

Chapter Six: Mitigation Strategies and Conclusions

6.1 Secure Diagnostics via SecOC

6.2 Intrusion Detection and Prevention Systems

6.3 Final Remarks and Future Work

Appendix A

Python Scripts

A.1 UDS Diagnostic Discovery

The complete script used to identify the Arbitration ID dedicated to Diagnostic is presented below:

Listing A.1: UDS Discovery

```
1 #CARINGCARIBOU DISCOVERY
2 import subprocess
3 from pathlib import Path
4 import can
5 import time
6
7 CURRENT_DIR = Path(__file__).resolve().parent
8 LOG_FILE_PATH = CURRENT_DIR / "logs/discovery_output.txt"
9 LOG_FILE_PATH_DIAGNOSTIC = CURRENT_DIR / "logs/diagnostic.txt"
10 #caringcaribou -i <interface> uds discovery -min <min_value> -max <
    max_value>
11
12 def verifica_coppia_diagnostica(client_id, server_id, interface="can0")
    :
13     """
14     send UDS DiagnosticSessionControl (10 01) to verify
15     if the couple CLIENT/SERVER is diagnostic related or false positive
16     .
17     """
18     is_extended = client_id > 0x7FF or server_id > 0x7FF
19     can_mask = 0xFFFFFFFF if is_extended else 0x7FF
20     can_filters = [{"can_id": server_id, "can_mask": can_mask, "
        extended": is_extended}]
21     SERVICE = 0x10
22     SUBFUNCTION = 0x01
23     try:
24         bus = can.interface.Bus(channel=interface, interface="socketcan
        ", can_filters=can_filters)
25         # Standard Payload Single Frame UDS (0x02 bytes, Service 0x10,
        Subfunction 0x01)
26         payload = [0x02, SERVICE, SUBFUNCTION, 0xAA, 0xAA, 0xAA, 0xAA,
        0xAA]
```

```

26         msg = can.Message(
27             arbitration_id=client_id,
28             data=payload,
29             is_extended_id=is_extended
30         )
31         bus.send(msg)
32         start_time = time.time()
33         while (time.time() - start_time) < 0.15:
34             rx_msg = bus.recv(timeout=0.05)
35             if rx_msg and rx_msg.arbitration_id == server_id:
36                 # If ECU responds with positive response (0x50) to
service 0x10
37                 #single frame
38                 if len(rx_msg.data) >= 3 and rx_msg.data[1] == (SERVICE
+ 0x40) and rx_msg.data[2] == SUBFUNCTION:
39                     bus.shutdown()
40                     return True
41                 #first frame
42                 if len(rx_msg.data) >= 4 and ((rx_msg.data[0] & 0xF0)
>> 4) == 0x01 and rx_msg.data[2] == (SERVICE + 0x40) and rx_msg.data
[3] == SUBFUNCTION:
43                     bus.shutdown()
44                     return True
45             bus.shutdown()
46         except Exception as e:
47             print(f"\n[!] Errore durante il test hardware della coppia: {e}
")
48         return False
49
50 def run_discovery(min_value, max_value, interface="can0"):
51     """
52     Discovery using caringcaribou.
53     """
54     coppie_candidate = []
55     discovered_diagnostics = []
56
57     # Strings to saveCaring Caribou log
58     stderr_log = ""
59     print("[*] Caring Caribou loading...")
60     # Ask Linux to use caringcaribou
61     process = subprocess.Popen(
62         ['caringcaribou', '-i', interface, "uds", "discovery", "-min",
str(min_value), "-max", str(max_value)],
63         stdout=subprocess.PIPE,
64         stderr=subprocess.PIPE,
65         text=True
66     )
67
68     # Capture standard output
69     for stdout_line in iter(process.stdout.readline, ""):
70         print(stdout_line, end="") # Show original output
71         if "0x" in stdout_line and "|" in stdout_line:
72             parti = stdout_line.split("|")

```

```

73         if len(parti) >= 3:
74             client_str = parti[1].strip()
75             server_str = parti[2].strip()
76             try:
77                 client_id = int(client_str, 16)
78                 server_id = int(server_str, 16)
79                 if (client_id, server_id) not in coppie_candidate:
80                     coppie_candidate.append((client_id, server_id))
81             except ValueError:
82                 continue
83
84     # Check for false positives
85     print(f"\n[*] Found {len(coppie_candidate)} couples. Validation... "
86 )
87
88     for client_id, server_id in coppie_candidate:
89         print(f"    [-] Test Client: 0x{client_id:08X} -> Server: 0x{
server_id:08X} ... ", end="", flush=True)
90         if verifica_coppia_diagnostics(client_id, server_id, interface)
91 :
92             print("VALID")
93             discovered_diagnostics.append((client_id, server_id))
94         else:
95             print("False Positive")
96
97     # Print on screen
98     print(f"| {'CLIENT ID':<12} | {'SERVER ID':<12} | ")
99     print("-"*31)
100     for c_id, s_id in discovered_diagnostics:
101         print(f"| 0x{c_id:08x} | 0x{s_id:08x} | ")
102     print("="*31)
103
104     # Print on file log
105     try:
106         with open(LOG_FILE_PATH, "w") as f:
107             f.write(f"| {'CLIENT ID':<12} | {'SERVER ID':<12} |\n")
108             for c_id, s_id in discovered_diagnostics:
109                 f.write(f"| 0x{c_id:08x} | 0x{s_id:08x} |\n")
110     except Exception as e:
111         print(f"[ERR] Errore during save: {e}")
112     return discovered_diagnostics
113
114 if __name__ == "__main__":
115     #discovery brute force
116     min_value = 0x00000000
117     max_value = 0xFFFFFFFF
118     result = run_discovery(min_value=min_value, max_value=max_value,
119                             interface=interface)
120     with open(LOG_FILE_PATH_DIAGNOSTIC, "a") as f:
121         for client_id, server_id in result:
122             f.write(f"{hex(client_id)} {hex(server_id)}\n")

```

Bibliography

- [1] SRM Technologies. *Vehicle Electronics Architecture – Past, Present, and Future*. <https://srmtech.com>. Accessed: 2026-07-03. 2025 (cit. on pp. 1, 3).
- [2] T. Nolte, H. Hansson, and L.L. Bello. “Automotive communications-past, current and future”. In: *2005 IEEE Conference on Emerging Technologies and Factory Automation*. Vol. 1. 2005, 8 pp.–992. DOI: 10.1109/ETFA.2005.1612631 (cit. on p. 1).
- [3] National Instruments. *Understanding CAN with Flexible Data Rate (CAN FD)*. 2022. URL: <https://ni.com> (visited on 07/03/2026) (cit. on p. 2).
- [4] Siemens AG. *Automotive Ethernet: High-Bandwidth Networking for Modern Vehicle Architectures*. 2024. URL: <https://www.siemens.com/it-it/technology/automotive-ethernet/> (visited on 07/03/2026) (cit. on p. 2).
- [5] International Organization for Standardization. *ISO/SAE 21434:2021: Road vehicles — Cybersecurity engineering*. 2021. URL: <https://iso.org> (cit. on p. 3).
- [6] Charlie Miller and Chris Valasek. *Remote Exploitation of an Unaltered Passenger Vehicle*. White Paper. IOActive Technical Research, 2015. URL: <https://illmatix.com/Remote%20Car%20Hacking.pdf> (visited on 07/03/2026) (cit. on p. 3).
- [7] Vector Informatik GmbH. *CAN Framing and Learning Modules*. <https://certification.vector.com/>. Accessed: 2026-07-03. 2021 (cit. on p. 4).

Dedications

Dedications for everyone LOOK dedications.tex